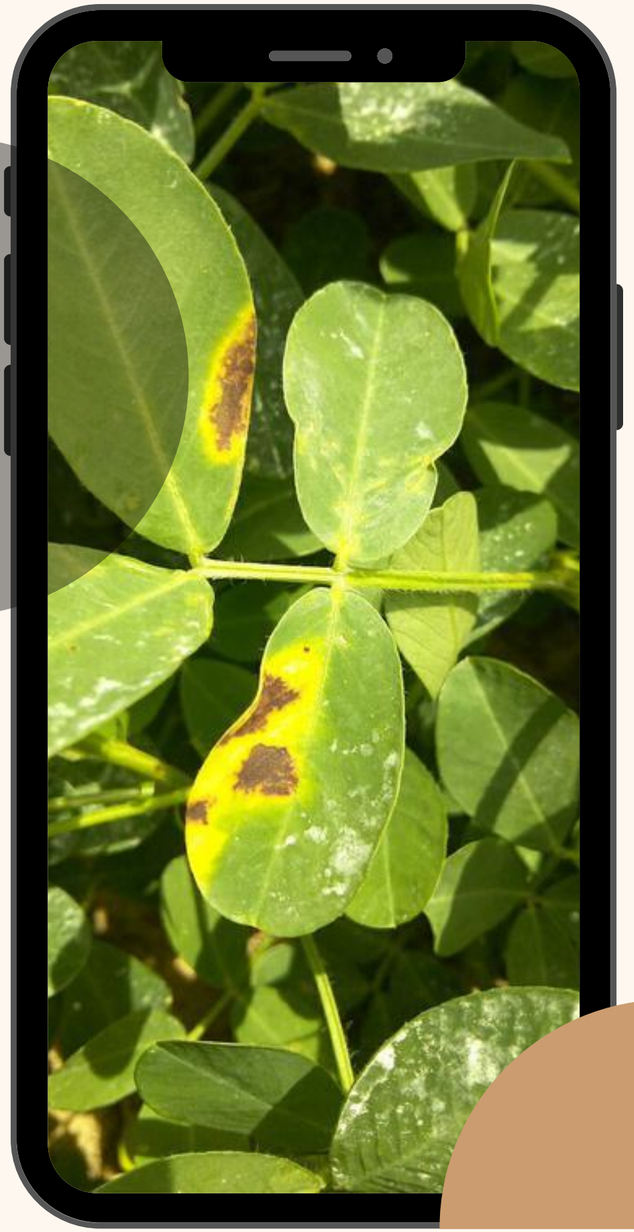
KLASIFIKASI PENYAKIT DAUN KACANG MENGGUNAKAN RANDOM FOREST BERDASARKAN FITUR WARNA DAN TEKSTUR

2318123 :HELENA RAHATEN

PENDAHULUAN

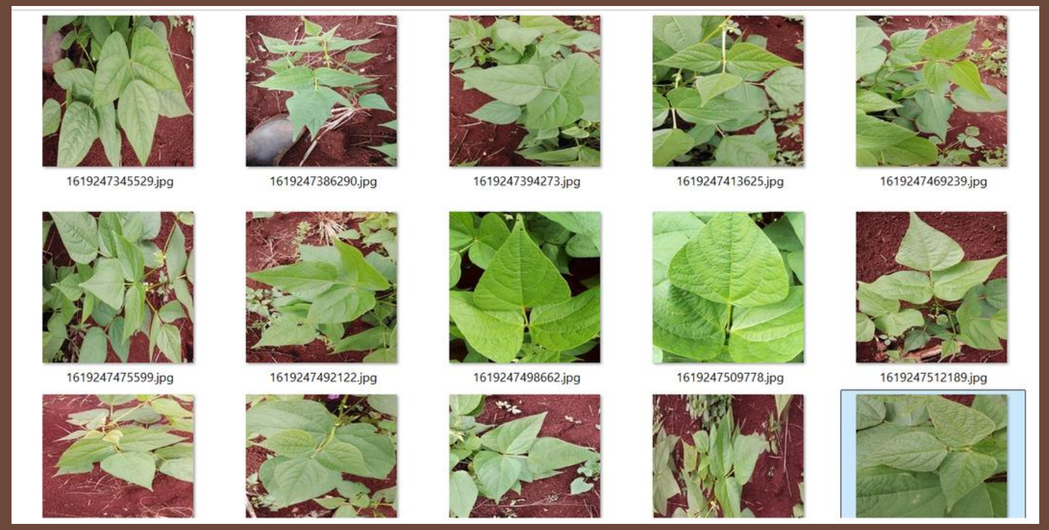
Laporan ini membahas implementasi machine learning untuk mengklasifikasikan penyakit pada daun kacang menggunakan algoritma Random Forest. Dataset yang digunakan berasal dari Kaggle dengan judul "Bean Crop Disease Diagnosis and Spatial Analysis" yang berisi gambar daun sehat, terserang Angular Leaf Spot (ALS), dan Bean Rust.



TUJUAN

Tujuan dari proyek ini adalah membangun model yang dapat mengenali jenis penyakit berdasarkan fitur warna dan tekstur dari citra daun. Hasil klasifikasi diharapkan dapat membantu petani dalam mendiagnosis penyakit tanaman secara cepat dan akurat.

DATASET



Dataset ini diunduh dari Kaggle dengan judul "Bean Crop Disease Diagnosis and Spatial Analysis". Berisi gambar daun kacang yang terbagi dalam empat kelas: Healthy (sehat), Angular Leaf Spot (ALS), Bean Rust (karat daun), dan Unknown. Dataset merupakan subset dari Makerere University Beans Image Dataset yang dikurasi untuk tugas klasifikasi penyakit tanaman.

LinkKaggle:https://www.kaggle.com/datasets/msjahid/bean-crop-disease-diagnosis-and-spatial-analysis

1. MOUNT GOOGLE DRIVE

```
from google.colab import drive
drive.mount('/content/drive')
```

Kode ini menghubungkan Google Colab dengan Google Drive agar program dapat mengakses dataset gambar. Setelah dijalankan, pengguna diminta mengklik link autentikasi dan memberikan izin akses. Fungsi utamanya adalah agar program dapat membaca dataset yang tersimpan di folder Tugas Uts/Classification serta menyimpan hasil output kembali ke Drive.

2.IMPORT LIBRARY

```
import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from imblearn.over_sampling import SMOTE
from tqdm import tqdm
import os
import warnings
warnings.filterwarnings('ignore')
```

Cell ini mengimpor semua modul yang diperlukan dalam satu blok. OpenCV digunakan untuk membaca dan memproses gambar, numpy dan pandas untuk manipulasi data numerik, sedangkan scikit-learn menyediakan algoritma Random Forest, LabelEncoder, StandardScaler, dan berbagai metrik evaluasi. Library imbalanced-learn diimpor untuk menggunakan SMOTE yang berfungsi menyeimbangkan data antar kelas, sementara matplotlib dan seaborn digunakan untuk visualisasi heatmap.

3.DISTRIBUSI JUMLAH DATA

```
train_path = "/content/drive/MyDrive/Tugas Uts/Classification/training"
test_path = "/content/drive/MyDrive/Tugas Uts/Classification/test"
```

Kode ini mendefinisikan lokasi folder tempat dataset gambar training dan test disimpan di Google Drive. train_path mengarah ke folder "training" yang berisi subfolder-subfolder sesuai kelas penyakit (Healthy, ALS, Bean Rust). test_path mengarah ke folder "test" yang berisi gambar untuk menguji performa model setelah proses training selesai

4.EKSREAKSI FITUR HSV DAN GLCM

```python
def extract(img_path):
    img = cv2.resize(cv2.imread(img_path), (128,128))
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    hist_h = cv2.calcHist([hsv], [0], None, [16], [0, 180]).flatten()
    hist_s = cv2.calcHist([hsv], [1], None, [16], [0, 256]).flatten()
    hist_v = cv2.calcHist([hsv], [2], None, [16], [0, 256]).flatten()
    warna = np.concatenate([hist_h, hist_s, hist_v])
    warna = warna / (np.sum(warna) + 1e-7)

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    tekstur = [np.mean(gray), np.std(gray),
               np.var(cv2.Laplacian(gray, cv2.CV_64F)),
               np.sum(cv2.Canny(gray,50,150) > 0) / (128*128)]

    return np.concatenate([warna, tekstur])  # 48 + 4 = 52 fitur
```

Fungsi extract() merupakan inti dari feature engineering yang mengubah gambar mentah menjadi vektor numerik 52 dimensi. Gambar dibaca, di-resize ke 128x128 piksel, lalu dikonversi ke ruang warna HSV untuk mengekstrak histogram 16 bins per channel menghasilkan 48 fitur warna. Selanjutnya, citra grayscale diproses untuk mendapatkan 4 fitur tekstur yaitu mean, standar deviasi, laplacian variance, dan edge density. Kombinasi kedua jenis fitur ini mampu merepresentasikan karakteristik visual penyakit daun seperti perubahan warna dan tekstur bercak.

5.EXPORT HASIL EKSTRAKSI KE CSV

```python
import pandas as pd

# Buat DataFrame
train_df = pd.DataFrame(X_train)
train_df['label'] = y_train
test_df = pd.DataFrame(X_test)
test_df['label'] = y_test

train_csv = f"{save_dir}/train_features_107.csv"
test_csv  = f"{save_dir}/test_features_107.csv"
train_df.to_csv(train_csv, index=False)
test_df.to_csv(test_csv, index=False)

print(f"✅ CSV training: {train_csv} (size: {train_df.shape})")
print(f"✅ CSV test: {test_csv} (size: {test_df.shape})")
```

Kode ini berfungsi menyimpan hasil ekstraksi fitur ke dalam format CSV. Array fitur X_train dan X_test dikonversi menjadi DataFrame pandas, kemudian ditambahkan kolom 'label' yang berisi kelas penyakit. File disimpan ke Google Drive dengan nama train_features_107.csv dan test_features_107.csv tanpa menyertakan index. Format CSV dipilih agar data dapat dibuka di Excel untuk analisis lebih lanjut atau digunakan kembali tanpa perlu mengekstrak fitur ulang.

6.PREPOCCESING

```
le = LabelEncoder()
y_train_enc = le.fit_transform(y_train)
y_test_enc = le.transform(y_test)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(X_train, y_train_enc)
y_pred = rf.predict(X_test)

akurasi = accuracy_score(y_test_enc, y_pred)
print(f"\n✅ AKURASI: {akurasi:.4f} ({akurasi*100:.2f}%)")
print("\n📋 CLASSIFICATION REPORT:")
print(classification_report(y_test_enc, y_pred, target_names=le.classes_))
```

LabelEncoder digunakan untuk mengubah label teks seperti "ALS" dan "Healthy" menjadi angka 0, 1, dan 2 karena model Random Forest hanya dapat memproses data numerik. Selanjutnya StandardScaler menstandarisasi setiap fitur agar memiliki rata-rata 0 dan standar deviasi 1, langkah penting karena fitur warna dan tekstur memiliki skala yang sangat berbeda. Proses ini memastikan bahwa tidak ada fitur yang mendominasi hanya karena memiliki nilai numerik yang lebih besar.

7.TRAINING RANDOM FOREST

```python
rf = RandomForestClassifier(
    n_estimators=300,
    max_depth=25,
    min_samples_split=4,
    min_samples_leaf=2,
    max_features='sqrt',
    random_state=42,
    n_jobs=-1
)

print("="*60)
print("🎯 Melatih Random Forest...")
print("="*60)
rf.fit(X_train_scaled, y_train_enc)
```

```python
# Prediksi
y_pred = rf.predict(X_test_scaled)
akurasi = accuracy_score(y_test_enc, y_pred)
print(f"\n✅ AKURASI PADA DATA TEST: {akurasi:.4f} ({akurasi*100:.2f}%)")
```

Model Random Forest dikonfigurasi dengan hyperparameter optimal yaitu 300 pohon keputusan untuk meningkatkan stabilitas prediksi dan kedalaman maksimal 25 untuk mencegah overfitting. Parameter min_samples_split=4 dan min_samples_leaf=2 mengontrol ukuran leaf node, sementara max_features='sqrt' menentukan jumlah fitur yang dipertimbangkan setiap split. Model dilatih menggunakan data training yang sudah melalui proses scaling, kemudian melakukan prediksi pada data test untuk menghitung akurasi akhir.

8.CONFUSION MATRIX

```python
from sklearn.metrics import classification_report, confusion_matrix

print("\n📋 CLASSIFICATION REPORT:")
print(classification_report(y_test_enc, y_pred, target_names=le.classes_))
```

```python
# Heatmap Confusion Matrix
plt.figure(figsize=(8,6))
cm = confusion_matrix(y_test_enc, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='RdYlGn',
            xticklabels=le.classes_, yticklabels=le.classes_,
            annot_kws={'size': 14})
plt.title(f'Confusion Matrix - Akurasi {akurasi:.2%}', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig(f"{save_dir}/confusion_matrix_improved.png", dpi=200)
plt.show()
print(f"✅ Heatmap disimpan di {save_dir}/confusion_matrix_improved.png")
```

Confusion matrix dibuat dengan membandingkan label asli dan label hasil prediksi menggunakan fungsi dari scikit-learn. Matriks 3x3 ini kemudian divisualisasikan sebagai heatmap dengan warna hijau untuk nilai tinggi dan merah untuk nilai rendah. Elemen diagonal menunjukkan jumlah prediksi yang benar untuk setiap kelas, sementara elemen di luar diagonal menunjukkan kesalahan prediksi seperti gambar ALS yang diprediksi sebagai Bean Rust. Heatmap disimpan sebagai file PNG untuk dokumentasi.

9.FEATURE IMPORTANCE

```python
# Nama fitur (96 warna + 11 tekstur)
feature_names = [f'warna_{i}' for i in range(96)] + \
                ['tekstur_mean', 'tekstur_std', 'tekstur_var', 'edge_density'] + \
                [f'glcm_k{k}_{s}' for k in [3,5,7] for s in ['mean','var']]

top_names = [feature_names[i] for i in indices]
top_imps = importances[indices]

plt.figure(figsize=(10,8))
sns.barplot(x=top_imps, y=top_names, palette='viridis')
plt.title('Top 30 Fitur Paling Berpengaruh', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig(f"{save_dir}/feature_importance_improved.png", dpi=200)
plt.show()
print(f"✅ Grafik pentingnya fitur disimpan di {save_dir}/feature_importance_improved.png")
```

Kode ini memanfaatkan atribut feature_importances_ dari Random Forest yang menyimpan nilai kepentingan setiap fitur dengan total keseluruhan 1. Nilai importance diambil, diurutkan dari tertinggi ke terendah menggunakan np.argsort(), lalu 20 fitur teratas dipilih untuk ditampilkan. Grafik bar horizontal yang dihasilkan menunjukkan fitur mana yang paling berpengaruh dalam pengambilan keputusan model, misalnya apakah fitur warna lebih dominan dibanding tekstur.

9.FEATURE IMPORTANCE

```
# Nama fitur (96 warna + 11 tekstur)
feature_names = [f'warna_{i}' for i in range(96)] + \
                ['tekstur_mean', 'tekstur_std', 'tekstur_var', 'edge_density'] + \
                [f'glcm_k{k}_{s}' for k in [3,5,7] for s in ['mean','var']]

top_names = [feature_names[i] for i in indices]
top_imps = importances[indices]

plt.figure(figsize=(10,8))
sns.barplot(x=top_imps, y=top_names, palette='viridis')
plt.title('Top 30 Fitur Paling Berpengaruh', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig(f"{save_dir}/feature_importance_improved.png", dpi=200)
plt.show()
print(f"✅ Grafik pentingnya fitur disimpan di {save_dir}/feature_importance_improved.png")
```

Kode ini memanfaatkan atribut feature_importances_ dari Random Forest yang menyimpan nilai kepentingan setiap fitur dengan total keseluruhan 1. Nilai importance diambil, diurutkan dari tertinggi ke terendah menggunakan np.argsort(), lalu 20 fitur teratas dipilih untuk ditampilkan. Grafik bar horizontal yang dihasilkan menunjukkan fitur mana yang paling berpengaruh dalam pengambilan keputusan model, misalnya apakah fitur warna lebih dominan dibanding tekstur.

10.TRAINING RANDOM FOREST

```
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
```

```
RandomForestClassifier
RandomForestClassifier(random_state=42)
```

Kode ini membuat model Random Forest dengan 100 pohon keputusan (n_estimators=100) dan random_state=42 untuk memastikan hasil yang konsisten setiap kali dijalankan. Model kemudian dilatih menggunakan data training (X_train sebagai fitur dan y_train sebagai label). Parameter random_state=42 berfungsi sebagai seed agar proses pengacakan dalam pembangunan pohon dapat direproduksi. Model versi ini menggunakan konfigurasi default tanpa tuning hyperparameter tambahan seperti max_depth atau min_samples_split.

11.SIMPAN HASIL SVC

```
# CELL 8: Menyimpan detail prediksi (label asli, prediksi, probabilitas)
results = pd.DataFrame({
    'True_Label': y_test,
    'Predicted_Label': le.inverse_transform(y_pred),
    'Correct': (y_test == le.inverse_transform(y_pred))
})
```

```
print("\n📊 Per kelas:")
per_class = results.groupby('True_Label')['Correct'].agg(['count', 'mean'])
per_class.columns = ['Jumlah', 'Akurasi_Kelas']
print(per_class)
```

DataFrame hasil prediksi dibangun dengan tiga kolom utama yaitu label asli, label hasil prediksi, dan status kebenaran (Correct). Selain itu, probabilitas keyakinan model untuk setiap kelas ditambahkan menggunakan metode predict_proba(), misalnya model 85% yakin suatu gambar adalah Healthy. File CSV terakhir disimpan ke Google Drive sebagai dokumentasi lengkap yang dapat dibuka di Excel

KESIMPULAN

Model Random Forest berhasil mengklasifikasikan penyakit daun kacang dengan akurasi 74.98% menggunakan fitur warna dan tekstur. Kelas yang paling sulit dikenali adalah ALS dan Bean Rust karena memiliki kemiripan bercak coklat pada daun.

TERIMAKASIH